BÁO CÁO ĐỒ ÁN

HỆ THỐNG NHÀ THÔNG MINH IoT TÍCH HỢP AI VÀ CẢNH BÁO AN NINH

Sinh viên: Nguyễn Hữu Tuấn Minh **Đề tài:** Xây dựng hệ thống Smart Home sử dụng ESP32, MQTT, Web Dashboard và ESP32-CAM **Phiên bản tài liệu:** 1.0 **Ngày cập nhật:** 12/11/2026

TÓM TẮT

Đề tài xây dựng một hệ thống nhà thông minh có khả năng thu thập dữ liệu cảm biến, điều khiển thiết bị điện, tự động hóa theo điều kiện môi trường và giám sát an ninh bằng camera. Hệ thống được chia thành ba cụm phần cứng ESP32. Cụm 1 xử lý chuyển động và cảnh báo, cụm 2 xử lý nhiệt độ, độ ẩm, ánh sáng và thiết bị đầu ra, còn ESP32-CAM chụp ảnh, điều khiển servo cửa và gửi sự kiện về máy chủ.

Các thiết bị trao đổi dữ liệu với Backend qua MQTT. Backend được xây dựng bằng FastAPI, lưu dữ liệu bằng SQLite/SQLAlchemy, cung cấp REST API và WebSocket cho giao diện Web. Khi phát hiện chuyển động, camera có thể tự chụp và gửi ảnh về Backend để lưu trữ, nhận diện khuôn mặt và phát cảnh báo có ảnh qua Telegram. Người dùng cũng có thể xem dữ liệu trực tiếp, lịch sử, thiết bị, kịch bản tự động hóa và điều khiển cửa từ giao diện Web.

MỤC LỤC

1. Giới thiệu đề tài
2. Mục tiêu và phạm vi
3. Danh sách linh kiện
4. Sơ đồ nối dây và phân bổ GPIO
5. Kiến trúc phần mềm
6. Cơ sở dữ liệu
7. Giao tiếp MQTT và API
8. Cấu trúc dự án

9. Các luồng hoạt động
10. Giao diện Web
11. Cài đặt và vận hành
12. Kiểm thử và đánh giá
13. Hạn chế và hướng phát triển

---

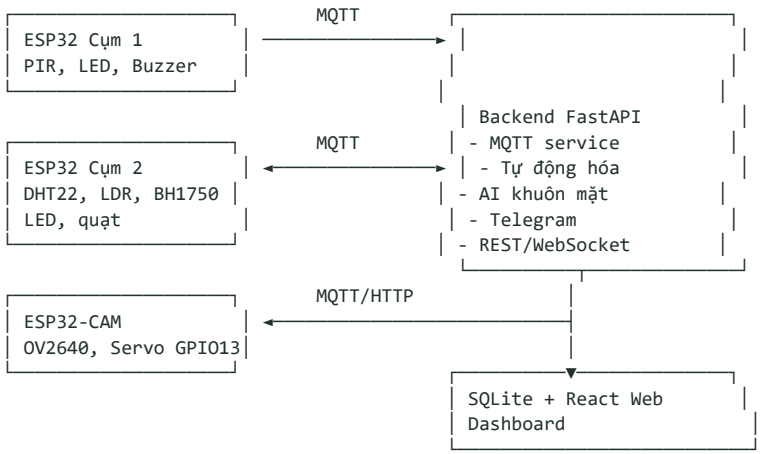
1. GIỚI THIỆU ĐỀ TÀI

1.1. Lý do chọn đề tài

Nhà ở hiện đại cần một hệ thống có thể theo dõi môi trường, điều khiển thiết bị từ xa và phát hiện sự kiện bất thường. Các vi điều khiển ESP32 có Wi-Fi, nhiều chân GPIO và chi phí thấp, phù hợp để xây dựng các node IoT. Tuy nhiên, nếu mỗi node hoạt động độc lập thì việc lưu lịch sử, phân quyền, tự động hóa và giám sát từ xa sẽ khó mở rộng.

Đề tài giải quyết vấn đề đó bằng cách tách hệ thống thành thiết bị nhúng, máy chủ trung tâm và giao diện Web. MQTT được dùng cho giao tiếp nhẹ giữa thiết bị và Backend; REST/WebSocket được dùng để kết nối Backend với Web.

1.2. Mô hình tổng quát



---

2. MỤC TIÊU VÀ PHẠM VI

2.1. Mục tiêu

- Đọc dữ liệu nhiệt độ, độ ẩm, cường độ sáng và chuyển động.
- Điều khiển đèn, còi, quạt và khóa cửa servo.
- Hiển thị dữ liệu trực tiếp và lịch sử trên Web Dashboard.
- Tự động hóa thiết bị theo ngưỡng cảm biến.
- Chụp ảnh bằng ESP32-CAM khi người dùng yêu cầu hoặc khi có chuyển động.
- Gửi ảnh sự kiện và cảnh báo an ninh qua Telegram.
- Lưu nhật ký cảm biến, lệnh điều khiển, nhận diện và cảnh báo.

- Phân quyền người dùng bằng JWT với các vai trò ADMIN, USER và GUEST.

2.2. Phạm vi

Hệ thống hiện chạy trong mạng LAN. Máy tính chạy Backend và Web dùng địa chỉ LAN của máy; các ESP32 kết nối cùng mạng Wi-Fi và gửi dữ liệu đến MQTT Broker/Backend. Cấu hình Wi-Fi, API key và Telegram được lưu trong các file cấu hình cục bộ, không đưa vào GitHub.

Chức năng giọng nói đã được loại bỏ khỏi giao diện hiện tại. Thư mục `mqtt-sample/` chỉ chứa công cụ mô phỏng và kiểm thử; khi triển khai với linh kiện thật, luồng hoạt động sử dụng ba firmware trong thư mục `firmware/`.

---

3. DANH SÁCH LINH KIỆN

3.1. Bộ điều khiển

Thiết bị	Số lượng	Vai trò
ESP32 DevKit/ESP32-WROOM	1	Cụm 1: PIR và thiết bị cảnh báo
ESP32 DevKit/ESP32-WROOM	1	Cụm 2: cảm biến môi trường và thiết bị đầu ra
AI Thinker ESP32-CAM	1	Chụp ảnh, gửi ảnh, điều khiển servo cửa
Máy tính chạy Backend	1	MQTT, API, cơ sở dữ liệu, AI, Telegram

3.2. Cảm biến và thiết bị của Cụm 1

Linh kiện	Chức năng
HC-SR501 PIR	Phát hiện chuyển động
LED	Báo hiệu khi có chuyển động hoặc theo lệnh Web
Active Buzzer	Phát cảnh báo bằng âm thanh
Điện trở và dây nối	Hoàn thiện mạch LED, nguồn và tín hiệu

Khi PIR phát hiện chuyển động, Cụm 1 bật LED và buzzer theo chế độ tự động, đồng thời phát bản tin lên topic `smarthome/security/motion` để ESP32-CAM chụp ảnh.

3.3. Cảm biến và thiết bị của Cụm 2

Linh kiện	Chức năng
DHT22	Đo nhiệt độ và độ ẩm
LDR/Module cảm biến ánh sáng	Đọc giá trị ADC để xác định sáng/tối
BH1750/GY-302	Đo cường độ sáng theo đơn vị lux
LED	Đèn chiếu sáng tự động hoặc điều khiển từ Web
Quạt 5V	Làm mát khi nhiệt độ cao và độ ẩm thấp
MOSFET/Relay module	Đóng cắt tải quạt, không cấp quạt trực tiếp từ GPIO

3.4. ESP32-CAM và cơ cấu cửa

Linh kiện	Chức năng
AI Thinker ESP32-CAM	Camera OV2640, Wi-Fi, HTTP/MQTT
Camera OV2640	Chụp ảnh sự kiện và ảnh nhận diện
Servo SG90	Mở/đóng cơ cấu khóa cửa
Nguồn 5V riêng cho servo	Giảm sụt áp và nhiễu khi servo khởi động
Dây GND chung	Tạo cùng mức tham chiếu giữa nguồn servo và ESP32-CAM

Servo dùng chân tín hiệu GPIO13. Dây nâu/đen nối GND, dây đỏ nối nguồn 5V phù hợp và dây cam/vàng nối GPIO13. Servo cần nguồn ổn định; không nên lấy dòng động cơ trực tiếp từ chân 3V3 của ESP32-CAM.

---

4. SƠ ĐỒ NỐI DÂY VÀ PHÂN BỔ GPIO

4.1. Cụm ESP32 1

Thiết bị	Tín hiệu	GPIO
PIR HC-SR501	OUT	27
LED	Điều khiển	26
Active Buzzer	Điều khiển	25

4.2. Cụm ESP32 2

Thiết bị	Tín hiệu	GPIO
DHT22	DATA	4
LED phòng	Điều khiển	2
Quạt qua MOSFET/Relay	Điều khiển	5
LDR	ADC	34
BH1750	SDA	22
BH1750	SCL	21

4.3. AI Thinker ESP32-CAM

Các chân camera OV2640 sử dụng theo sơ đồ chuẩn AI Thinker. Chân servo được đặt riêng tại GPIO13, là chân còn trống khi không sử dụng giao tiếp thẻ nhớ SD trong cấu hình hiện tại.

Chức năng	GPIO
Servo SG90	13
Flash LED	4
Camera PWDN	32
Camera XCLK	0
Camera SIOD/SIOC	26/27

Chức năng	GPIO
Camera dữ liệu	5, 18, 19, 21, 34, 35, 36, 39
Camera đồng bộ	VSYNC 25, HREF 23, PCLK 22

GPIO camera là các chân dành riêng cho module OV2640 và không được dùng tùy tiện cho tải ngoài. Mạch servo phải có nguồn và GND chung với ESP32-CAM.

---

5. KIẾN TRÚC PHẦN MỀM

5.1. Các lớp chính

1. **Lớp thiết bị:** firmware Arduino chạy trên ESP32, đọc GPIO/cảm biến, điều khiển đầu ra và giao tiếp MQTT.

2. **Lớp truyền thông:** MQTT Broker nhận và chuyển tiếp message giữa các node; Backend dùng Paho-MQTT để publish/subscribe.

3. **Lớp nghiệp vụ:** FastAPI xử lý xác thực, dữ liệu cảm biến, điều khiển, tự động hóa, nhận diện ảnh và cảnh báo.

4. **Lớp lưu trữ:** SQLite kết hợp SQLAlchemy lưu trạng thái, lịch sử và cấu hình.

5. **Lớp giao diện:** React + TypeScript + Vite gọi REST API và nhận sự kiện WebSocket.

6. **Lớp thông báo:** Telegram Bot API gửi cảnh báo văn bản hoặc ảnh sự kiện.

5.2. Công nghệ sử dụng

Thành phần	Công nghệ
Firmware	PlatformIO, Arduino Framework, C++
MQTT trên firmware	PubSubClient, ArduinoJson
Backend	Python, FastAPI, Uvicorn
MQTT trên Backend	Paho-MQTT và broker nhúng asyncio
Cơ sở dữ liệu	SQLite WAL, SQLAlchemy 2.0
Nhận diện ảnh	OpenCV Haar Cascade và so khớp ảnh mẫu
Xác thực	JWT, bcrypt, RBAC
Frontend	React, TypeScript, Vite
Biểu đồ	Recharts
Thông báo	Telegram Bot API

---

6. CƠ SỞ DỮ LIỆU

Database mặc định là SQLite tại `backend/data/smarthome.db`. Backend tự tạo bảng khi khởi động và thêm dữ liệu ban đầu nếu database chưa có dữ liệu.

6.1. Mô hình dữ liệu

`users`

```
│
├── device_history ──── devices
├── sensor_history
├── recognition_history ──── captured_events/*.jpg
├── automation_rules
├── alerts
└── system_settings
```

6.2. Các bảng chính

Bảng	Dữ liệu lưu trữ
`users`	Tài khoản, mật khẩu đã băm, họ tên, vai trò và trạng thái hoạt động
`devices`	Thiết bị đầu ra, cụm ESP32, loại thiết bị, GPIO, trạng thái, chế độ và online
`sensor_history`	Nhiệt độ, độ ẩm, ADC/Lux, PIR và trạng thái đèn/quạt/còi theo thời gian
`device_history`	Lệnh điều khiển, command ID, thiết bị đích, ACK, kết quả và nguồn kích hoạt
`recognition_history`	Event ID, kết quả `KNOWN/UNKNOWN/NO_FACE`, confidence, ảnh và trạng thái cửa
`automation_rules`	Tên kịch bản, cụm thiết bị, bật/tắt và cấu hình JSON
`alerts`	Loại cảnh báo, mức độ, nội dung, ảnh, trạng thái đã đọc và Telegram
`system_settings`	Ngưỡng cảm biến, trạng thái bảo vệ và cấu hình Telegram

6.3. Dữ liệu ảnh

- `backend/data/owner_faces/owner.jpg`: ảnh mẫu khuôn mặt chủ nhà.
- `backend/data/captured_events/<event_id>.jpg`: ảnh chụp của từng sự kiện.
- Ảnh được phục vụ qua endpoint `/api/security/images/{filename}`.
- Token Telegram, mật khẩu Wi-Fi, API key và chat ID chỉ nằm trong `.env` hoặc `secrets.h` cục bộ.

6.4. Thiết bị được seed

ID	Cụm	GPIO	Loại
`light_1`	`esp32_cluster1`	26	LED cảnh báo
`buzzer_1`	`esp32_cluster1`	25	Còi
`light_2`	`esp32_cluster2`	2	LED phòng
`fan_1`	`esp32_cluster2`	5	Quạt
`door_servo_1`	`esp32_cam_01`	13	Servo cửa

---

7. GIAO TIẾP MQTT VÀ API

7.1. Các topic MQTT

Topic	Hướng	Nội dung
`smarthome/sensor/cluster1`	Cụm 1 → Backend	Dữ liệu PIR và trạng thái đèn/còi
`smarthome/security/motion`	Cụm 1 → ESP32-CAM/Backend	Sự kiện có chuyển động
`smarthome/device/cluster1/command`	Backend → Cụm 1	Lệnh bật/tắt đèn, còi, chế độ

Topic	Hướng	Nội dung
`smarthome/device/cluster1/ack`	Cụm 1 → Backend	ACK lệnh Cụm 1
`smarthome/sensor/cluster2`	Cụm 2 → Backend	Nhiệt độ, độ ẩm, ánh sáng
`smarthome/device/cluster2/command`	Backend → Cụm 2	Lệnh thiết bị Cụm 2
`smarthome/device/cluster2/ack`	Cụm 2 → Backend	ACK lệnh Cụm 2
`smarthome/security/access/esp32_cam_01/command`	Backend → ESP32-CAM	Lệnh cửa, mở/đóng và capture
`smarthome/security/access/esp32_cam_01/ack`	ESP32-CAM → Backend	ACK servo/camera
`smarthome/security/recognition`	ESP32-CAM/Backend → Backend	Kết quả nhận diện
`smarthome/device/.../status`	Thiết bị → Backend	Trạng thái online và thiết bị
`smarthome/system/.../status`	Thiết bị → Backend	Trạng thái hệ thống/firmware

Một message điều khiển thường có `command_id`, `device_id`, `state`, `mode` và có thể có object `outputs`. Backend chỉ xem lệnh hoàn tất sau khi nhận ACK hoặc status tương ứng.

7.2. API tiêu biểu

Method	Endpoint	Chức năng
`POST`	`/api/auth/login`	Đăng nhập và nhận JWT
`GET`	`/api/health`	Kiểm tra Backend
`GET`	`/api/sensors/latest`	Dữ liệu cảm biến mới nhất
`GET`	`/api/sensors/history`	Lịch sử để vẽ biểu đồ
`GET`	`/api/devices`	Danh sách và trạng thái thiết bị
`POST`	`/api/devices/control`	Điều khiển thiết bị
`GET/POST/PUT/DELETE`	`/api/automation/rules`	Quản lý kịch bản tự động hóa
`POST`	`/api/mqtt/cam/capture`	Yêu cầu ESP32-CAM chụp ảnh
`POST`	`/api/mqtt/cam/door`	Điều khiển cửa từ Web
`POST`	`/api/security/recognize`	Nhận ảnh và nhận diện khuôn mặt
`POST`	`/api/security/enroll`	Lưu ảnh mẫu chủ nhà
`GET`	`/api/recognitions`	Lịch sử nhận diện
`GET`	`/api/alerts`	Danh sách cảnh báo
`GET/PUT`	`/api/settings`	Đọc/cập nhật cấu hình
`WS`	`/ws/dashboard`	Cập nhật thời gian thực cho Web

---

8. CẤU TRÚC DỰ ÁN

```
IOT/
├── backend/
│   ├── app/
│   │   ├── api/
```

```
│   │   │   ├── auth.py              # Đăng nhập, người dùng, phân quyền
│   │   │   ├── devices.py           # Điều khiển thiết bị
│   │   │   ├── sensors.py           # Dữ liệu cảm biến
│   │   │   ├── mqtt_routes.py       # MQTT state/log/control
│   │   │   ├── security_ai.py       # Ảnh, AI và cửa servo
│   │   │   ├── automation.py        # Kịch bản tự động hóa
│   │   │   └── alerts_settings.py   # Cảnh báo và cài đặt
│   │   ├── core/
│   │   │   ├── config.py            # Biến môi trường và đường dẫn
│   │   │   ├── database.py          # SQLAlchemy/SQLite
│   │   │   └── security.py          # JWT, bcrypt, RBAC
│   │   ├── models/models.py         # 8 bảng và dữ liệu seed
│   │   ├── schemas/schemas.py       # Pydantic schema
│   │   ├── services/
│   │   │   ├── mqtt_service.py      # Publish/subscribe MQTT
│   │   │   ├── automation.py        # Bộ máy tự động hóa
│   │   │   ├── face_recognition.py  # So khớp khuôn mặt
│   │   │   ├── telegram_bot.py      # Gửi Telegram
│   │   │   └── broker.py             # MQTT broker nhúng
│   │   ├── websocket/manager.py     # Quản lý WebSocket
│   │   └── main.py                  # Khởi tạo FastAPI
│   ├── data/                        # Database và ảnh sự kiện
│   ├── tests/test_api.py            # Kiểm thử API
│   ├── .env.example                 # Cấu hình mẫu, không chứa bí mật
│   ├── requirements.txt
│   └── run.py
├── frontend/
│   ├── src/
│   │   ├── components/              # Layout và component dùng chung
│   │   ├── pages/                   # Dashboard, Devices, Security...
│   │   ├── api.ts                   # Axios API client
│   │   ├── LiveContext.tsx           # WebSocket/polling thời gian thực
│   │   └── types.ts                  # Kiểu dữ liệu TypeScript
│   ├── package.json
│   └── vite.config.ts
├── firmware/
│   ├── esp32-cluster1-test/
│   │   ├── include/pins.h
│   │   ├── include/secrets.example.h
│   │   └── src/main.cpp
│   ├── esp32-cluster2-test/
│   │   ├── include/pins.h
│   │   ├── include/secrets.example.h
│   │   └── src/main.cpp
│   └── esp32-cam/
│       ├── include/pins.h
│       ├── include/secrets.example.h
│       └── src/main.cpp
├── mqtt-sample/                     # Công cụ mô phỏng/kiểm thử tùy chọn
├── API_SPEC.md
├── MQTT_SPEC.md
├── WEB_DASHBOARD_SPEC.md
└── README.md
```

---

9. CÁC LUỒNG HOẠT ĐỘNG

9.1. Luồng dữ liệu cảm biến

1. Firmware Cụm 1 hoặc Cụm 2 đọc cảm biến theo chu kỳ.
2. Firmware tạo payload JSON gồm ID thiết bị, giá trị cảm biến, trạng thái và thời gian.
3. Firmware publish message lên topic cảm biến tương ứng.
4. Backend MQTT service nhận message, cập nhật snapshot, lưu `sensor_history` và phát sự kiện WebSocket.
5. Frontend nhận dữ liệu qua WebSocket; nếu WebSocket gián đoạn, frontend chuyển sang gọi API định kỳ.

9.2. Luồng điều khiển thiết bị từ Web

```
Người dùng bấm nút
        ↓
React gọi REST API
        ↓
FastAPI kiểm tra JWT/quyền và tạo command_id
        ↓
MQTT publish command
        ↓
ESP32 nhận lệnh và ghi GPIO
        ↓
ESP32 publish ACK/status
        ↓
Backend lưu device_history và gửi WebSocket
        ↓
Web hiển thị trạng thái đã xác nhận
```

9.3. Luồng tự động hóa môi trường

• **Cụm 1:** PIR có chuyển động → bật LED và buzzer theo thời gian giữ cấu hình → publish sự kiện chuyển động.

• **Cụm 2:** nhiệt độ từ 30°C trở lên và độ ẩm không vượt ngưỡng 40% → bật quạt; khi điều kiện không còn đúng → tắt quạt.

• **Cụm 2:** LDR có ADC nhỏ hơn 2000 hoặc lux thấp hơn 50 → bật đèn; khi đủ sáng → tắt đèn.

Các giá trị ngưỡng có thể được lưu trong `automation_rules` và `system_settings`.

9.4. Luồng chuyển động, camera và Telegram

```
PIR Cụm 1 phát hiện chuyển động
        ↓ MQTT smarthome/security/motion
ESP32-CAM nhận sự kiện
        ↓ chụp ảnh OV2640
POST ảnh JPEG /api/security/recognize
        ↓
Backend lưu captured_events/<event_id>.jpg
        ↓ OpenCV đối chiếu ảnh mẫu owner.jpg
Lưu recognition_history
        ↓
Tạo alert MOTION_CAPTURE kèm image_url
        ↓
Telegram Bot API gửi ảnh + nội dung cảnh báo
```

Ảnh chuyển động được gửi về Telegram nhằm giám sát an ninh. Việc mở cửa từ mục **Thiết bị** là thao tác điều khiển trực tiếp; mục **AI & An ninh** dùng để chụp ảnh và xác minh khi người dùng yêu cầu.

9.5. Luồng điều khiển cửa

- Lệnh `OPEN` làm servo quay đến góc mở.
- Lệnh `CLOSE` làm servo quay về góc khóa.
- Trạng thái cửa được giữ nguyên cho đến khi có lệnh mới; không tự động đóng sau 5 giây.
- ESP32-CAM trả ACK và status về Backend.
- Cơ chế nhận diện khuôn mặt vẫn tạo lịch sử `KNOWN`, `UNKNOWN` hoặc `NO_FACE` và có thể phát cảnh báo, nhưng điều khiển trực tiếp trong danh mục thiết bị không cần xác thực khuôn mặt.

---

10. GIAO DIỆN WEB

Frontend gồm các khu vực chính:

Trang	Chức năng
Tổng quan	Hiển thị tình trạng hệ thống, cảm biến mới nhất và biểu đồ
Thiết bị	Xem online/offline, bật/tắt đèn, còi, quạt và mở/đóng cửa
Tự động hóa	Bật/tắt và chỉnh các kịch bản
AI & An ninh	Chụp ảnh, xem ảnh đầy đủ khung, enroll mặt chủ nhà và lịch sử nhận diện
Lịch sử	Xem dữ liệu cảm biến, lệnh điều khiển và sự kiện
Cài đặt	Ngưỡng tự động, bảo vệ và Telegram
Người dùng	Quản lý tài khoản theo quyền

Giao diện đã sử dụng UTF-8 cho tiếng Việt. Ảnh camera dùng tỷ lệ 4:3 và `object-fit: contain` để tránh cắt mất nội dung ảnh.

---

11. CÀI ĐẶT VÀ VẬN HÀNH

11.1. Khởi động Backend

```
cd C:\Users\Minh\Desktop\Hoc\IOT\backend
py -3 -m pip install -r requirements.txt
py -3 run.py
```

Địa chỉ kiểm tra:

- API: `http://localhost:8000`
- Swagger: `http://localhost:8000/docs`
- Health: `http://localhost:8000/api/health`
- WebSocket: `ws://localhost:8000/ws/dashboard`

11.2. Khởi động Frontend

```
cd C:\Users\Minh\Desktop\Hoc\IOT\frontend
npm install
npm run dev
```

Sau đó mở địa chỉ Vite hiển thị trên terminal, thường là `http://localhost:5173`.

11.3. Build và nạp firmware

```
# ESP32-CAM
cd C:\Users\Minh\Desktop\Hoc\IOT\firmware\esp32-cam
py -3 -m platformio run -t upload
py -3 -m platformio device monitor -b 115200

# Cụm 1
cd ..\esp32-cluster1-test
py -3 -m platformio run -t upload

# Cụm 2
cd ..\esp32-cluster2-test
py -3 -m platformio run -t upload
```

Mỗi firmware cần có file `include/secrets.h` cục bộ dựa trên `secrets.example.h`. File thật được Git ignore để không làm lộ mật khẩu Wi-Fi, địa chỉ riêng hoặc API key.

11.4. Kiểm tra kết nối

1. Kiểm tra máy tính và các ESP32 cùng mạng Wi-Fi.

2. Mở Serial Monitor của từng board ở tốc độ 115200.
3. Kiểm tra Backend báo MQTT connected.
4. Kiểm tra `GET /api/mqtt/state`.
5. Kiểm tra status của ESP32-CAM và thử `/status` hoặc `/capture` trên IP của camera.
6. Thử PIR, quan sát log `motion trigger received`, `camera captured` và HTTP 200 của Backend.

---

12. KIỂM THỬ VÀ ĐÁNH GIÁ

12.1. Kiểm thử chức năng

Mã	Nội dung	Kết quả mong đợi
TC01	Đăng nhập bằng tài khoản hợp lệ	Backend trả JWT và Web vào Dashboard
TC02	Đọc dữ liệu Cụm 2	Nhiệt độ, độ ẩm, ADC/lux hiển thị đúng
TC03	PIR phát hiện chuyển động	LED/còi hoạt động và topic motion được publish
TC04	Điều khiển LED/quạt từ Web	Thiết bị nhận command, trả ACK và Web cập nhật
TC05	Bấm mở/đóng cửa	Servo quay đúng góc và giữ trạng thái đến lệnh tiếp theo
TC06	Bấm chụp trong AI & An ninh	Web nhận ảnh và hiển thị đủ khung
TC07	PIR kích hoạt camera	ESP32-CAM tự chụp, Backend lưu ảnh và tạo alert
TC08	Bật Telegram	Telegram nhận cảnh báo, sự kiện ảnh nhận đúng
TC09	Mất MQTT	Web hiển thị offline hoặc trạng thái chưa xác nhận
TC10	Người dùng GUEST điều khiển	Backend từ chối thao tác không đủ quyền

12.2. Kiểm thử phần mềm đã thực hiện

- Build Frontend bằng `npm run build`.
- Build firmware ESP32-CAM bằng PlatformIO.
- Biên dịch các module Python Backend.
- Kiểm tra API camera và trạng thái `camera_ready`, `mqtt_connected`.
- Kiểm tra file bí mật không nằm trong commit GitHub.

12.3. Tiêu chí đánh giá

Hệ thống được xem là hoạt động đạt yêu cầu khi dữ liệu đi từ cảm biến đến Database và Web, lệnh điều khiển nhận được ACK, sự kiện chuyển động tạo ảnh, ảnh được lưu và cảnh báo Telegram được gửi thành công.

---

13. HẠN CHẾ VÀ HƯỚNG PHÁT TRIỂN

13.1. Hạn chế

- Hệ thống hiện phụ thuộc mạng LAN và máy tính chạy Backend.
- ESP32-CAM có tài nguyên hạn chế; nhận diện khuôn mặt được xử lý tại Backend.

- Servo và tải quạt cần nguồn riêng ổn định để tránh reset ESP32.
- SQLite phù hợp cho mô hình cá nhân/nhóm nhỏ; chưa tối ưu cho nhiều người dùng đồng thời.
- Ngưỡng cảm biến phụ thuộc điều kiện lắp đặt thực tế và cần hiệu chỉnh.

13.2. Hướng phát triển

- Đóng gói Backend và MQTT bằng Docker Compose.
- Chuyển Database sang PostgreSQL khi triển khai nhiều người dùng.
- Thêm HTTPS, quản lý API key theo thiết bị và cơ chế refresh token.
- Thêm hàng đợi xử lý ảnh và lưu ảnh trên object storage.
- Hiệu chỉnh cảm biến tự động và bổ sung biểu đồ thống kê dài hạn.
- Thêm OTA cho firmware ESP32.
- Bổ sung kiểm thử tự động cho MQTT, WebSocket và luồng Telegram.
- Có thể bổ sung dashboard cho nhiều ngôi nhà hoặc nhiều camera.

KẾT LUẬN

Đề tài đã xây dựng được một mô hình Smart Home hoàn chỉnh gồm phần cứng ESP32, giao thức MQTT, Backend FastAPI, cơ sở dữ liệu SQLite, giao diện React và hệ thống camera/AI. Kiến trúc phân lớp giúp các node cảm biến, thiết bị đầu ra, camera và giao diện Web hoạt động độc lập nhưng vẫn đồng bộ qua Backend. Hệ thống đáp ứng các chức năng chính: giám sát môi trường, điều khiển thiết bị, tự động hóa, quản lý cửa servo, lưu lịch sử và gửi cảnh báo an ninh kèm ảnh qua Telegram.

Tài liệu này được viết theo mã nguồn thực tế trong repository và có thể dùng làm báo cáo nền để bổ sung ảnh mạch, ảnh giao diện, kết quả đo và thông tin thành viên trước khi nộp.